

Rescue Line Team BIGG-IRMI

Technical Description Paper

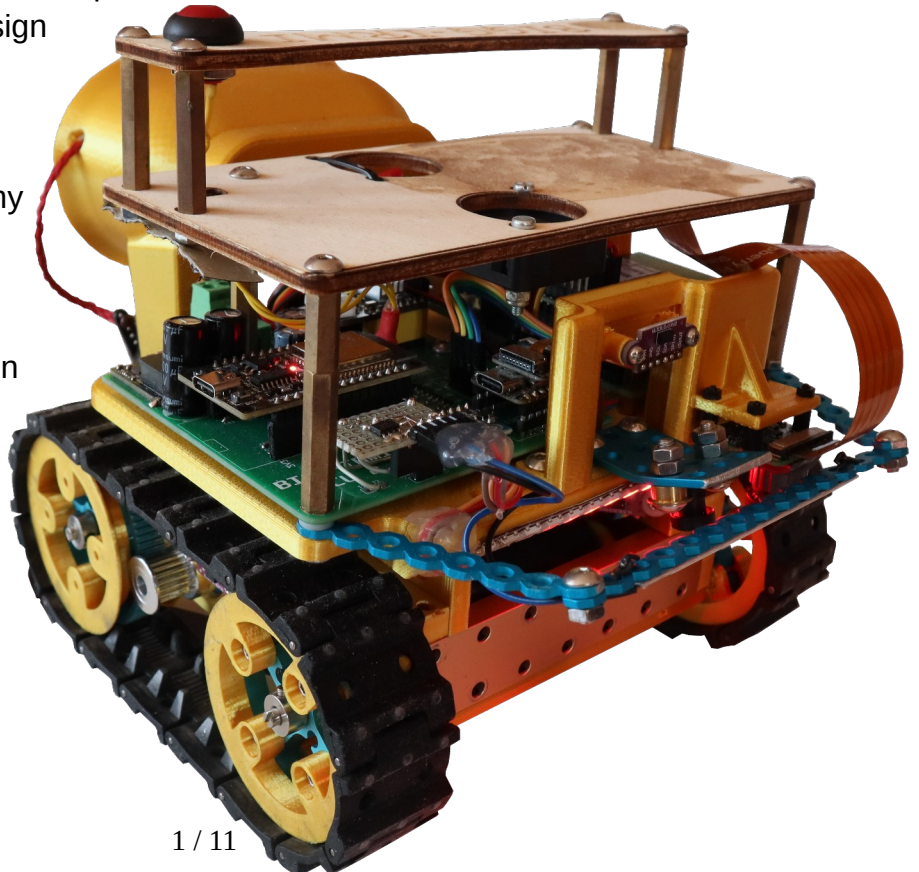


Iren Biggel

- Born in 2008, Germany
- Team Captain
- Embedded Software Development
- Hardware & PCB Design
- Project Management

Mila Biggel

- Born in 2011, Germany
- Team Member
- Mechanical CAD
- 3D Manufacturing
- Media content creation
- Testing



Project Planning – from Design to Deployment

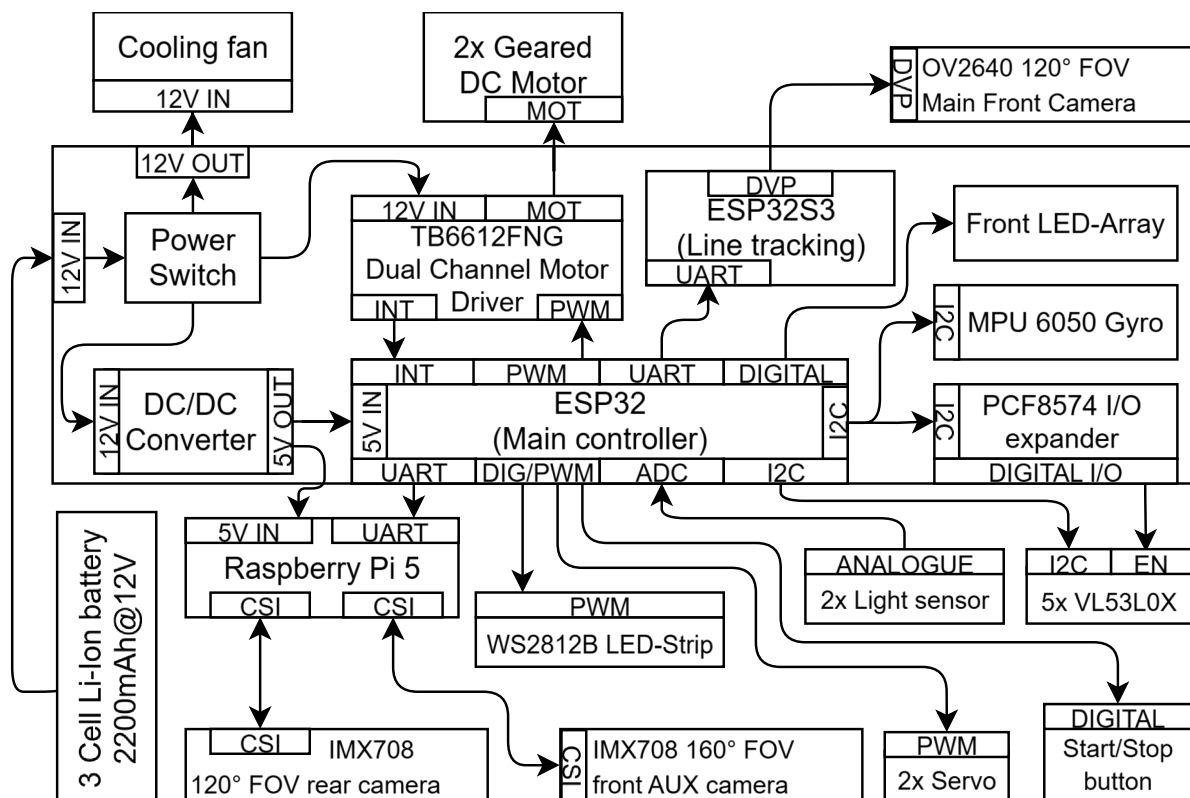
Requirements definition

Our robot must be able to follow lines and turn right at intersections. It must be able to navigate obstacles such as ramps, seesaws, bumpers and debris. Furthermore, it should be able to recognise the evacuation zone, locate all victims, and deposit them correctly in the colour-coded corners. It should then exit the evacuation zone correctly and stop at the goal tile. All encountered difficulties or obstacles should be easily overcome.

Overall Project Plan

As we already have a decent chassis from last year's robot, we can reuse most of it. However, as this year's robot needs to be improved in many ways, we will then need to select an appropriate power supply, suitable sensors and camera modules, and mount them securely and usefully to the chassis. Next, we will select the appropriate electronic components, e.g. motor drivers, microcontrollers, voltage regulators and HIDs, and position them within the chassis. After that, we will design a PCB to connect most of the selected components. This will be followed by the development of software for the selected components. It is imperative that we carry out regular tests to ensure correct function.

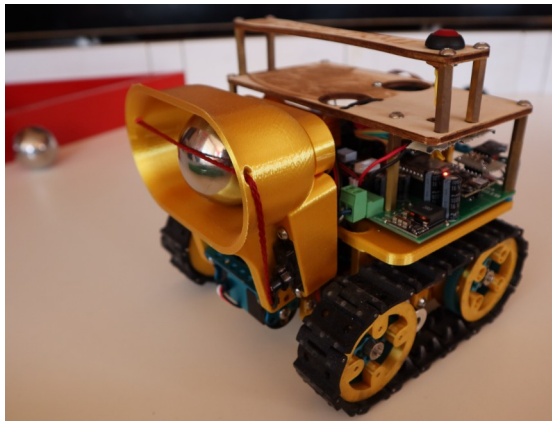
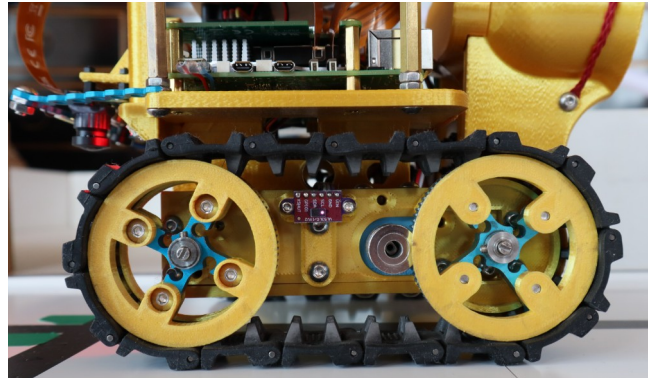
Integration Plan / System Engineering



- 3D FDM printed chassis:
 - Individual customisation
 - Optimum dimensions for the course
 - Weight saving
 - Driven by rubber tracks for optimum traction and to overcome all obstacles/debris.
- Power supply:
 - 3 cell Li-Ion battery with 2200mAh at 12V
 - Quick-charging capability
- Main-Controller:
 - ESP32 with all needed I/O capabilities
 - At least 2 UART interfaces to communicate with other hardware
- Front camera, communicates with the controller via UART and enables line tracking including correct turning at intersections
- Front camera #2 & Rear camera, communicates with the main controller via UART. Using AI, it checks for evacuation zone entrances/exits and assists with turning decisions at intersections. It also enables AI-based victim detection and AI-supported victim depositing.
- Adafruit VL53L0X time of flight distance sensors, connected via our I2C-Bus
- WS2812b LED strips, controlled via digital PWM to ensure good lighting for our cameras during runs. Additionally a 5W LED module is mounted to ensure consistent lighting conditions during competitions.
- Light sensors for victim presence detection runs via analogue
- All servos are also controlled via digital PWM
- Dual channel motor driver with encoder is connected to the main controller

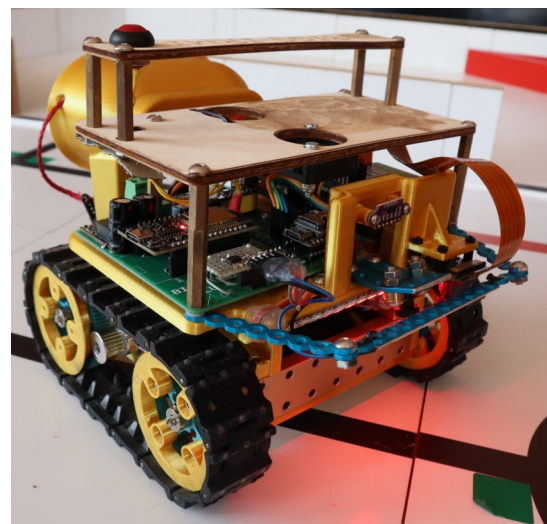
Mechanical design and manufacturing

The majority of our robot is composed of 3D-printed components, enabling us to tailor them to our specific requirements. We use rubber tracks to ensure optimal traction and manoeuvrability. These components are driven via a gearbox and encoder motors, ensuring the motors operate within the optimal speed range. In order to achieve a low centre of gravity, we placed the battery and motors at the bottom of the robot. In order to guarantee an unobstructed view of the front camera, we have strategically positioned the victim grabber device at the rear of the robot. As with the chassis, the component has been manufactured using 3D printing. The victim grabber device consists of a large victim container and a holding mechanism consisting of a cord that can be tensioned and



released by a servo. Furthermore, the robot is equipped with light sensors that accurately determine the number of victims present in the victim-grabber-device. Carrying out a conductivity test is not necessary as our AI-supported victim rescue system is able to differentiate between alive and dead victims. The device is designed to rescue the victims by moving towards the floor, while the cord servo tightens the cord. At the point at which the servo has moved up, the victim is picked up. The existing hardware is then used to position the victims in their respective corners. To facilitate the identification of corners, we utilise our

AI-based victim recognition system. The large servo is responsible for moving the entire victim grabber downwards, while the cord servo is responsible for releasing the victims. The rear camera is mounted low to improve victim detection. Since the battery had already been installed in the lower area, the remaining components, as well as the front cameras for tracking the line and detecting the entrance and exit of the evacuation zone, had to be installed in the upper part of the robot. Rather than mounting the cameras in the centre facing downwards as would be the common sense approach, they are mounted high at the front of the robot. This enables the robot to see ahead of its current position, allowing it to prepare for or react to oncoming intersections or turns, and ensuring smooth line following. To securely connect all electronic components, a custom PCB was manufactured. In order to protect the electronic components we have installed a laser-cut wooden interlayer. This interlayer also serves the function of mounting the cooling fan for the microcontroller and motor driver, as these components generate a significant amount of heat during normal operation. The handle and the start/stop button have been mounted in the desired location.



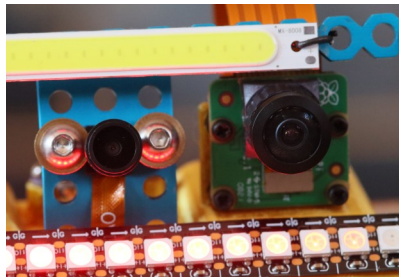
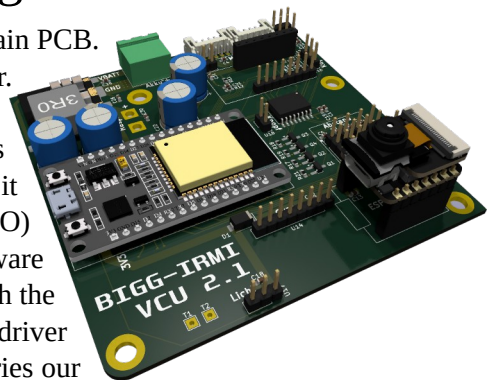
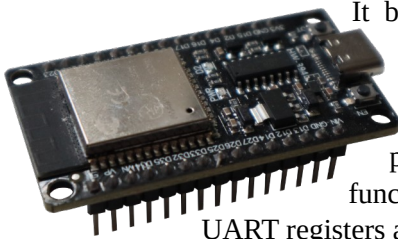
Electronic design and manufacturing

The entire electronic design of the robot is centred around our main PCB.

It brings all the components together.

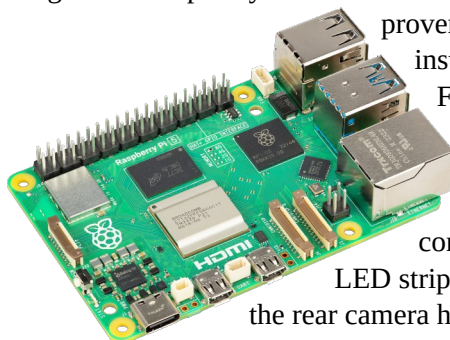
Our primary controller is the ESP32. We chose it because it is reliable and powerful, and it provides all the input/output (I/O) functionality that we require. Two hardware

UART registers are required for communication with the auxiliary controllers. The PCB also carries our motor driver (TB6612FNG) and our I2C I/O expander (PCF8574). It also carries our gyroscope sensor (MPU6050) and the driver circuitry that enables and disables our distance sensors.



The main controller communicates with the front camera module to facilitate line tracking. This module comprises one OV2640 camera with a 120° field of view (FOV) and one IMX708 camera with a 160° FOV, with one camera connected to an ESP32S3 microcontroller. The second camera is connected to a Raspberry Pi 5, which runs local, self-trained AI models to assist in detecting the silver tape at the entrance

to the evacuation zone. The Raspberry Pi also assists the line-tracking camera in detecting green dots at intersections, as the OV2640 has a very limited colour range. The Raspberry Pi is also connected to the main controller via UART. This setup has



proven to be a highly reliable line-tracking system. We installed a Raspberry Pi Camera Module 3 with a 120°

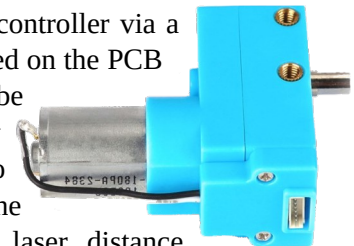
FOV at the rear, which is connected to the Raspberry Pi 5. The Pi also uses self-trained local AI to accurately locate victims and evacuation points within the evacuation zone. In order to ensure that environmental conditions do not obstruct the camera's view, addressable WS2812B



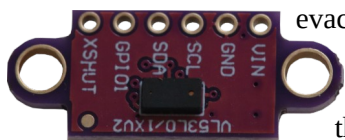
LED strips and a bright white LED array have been installed at the front. As the rear camera has such wide coverage, no lights have been mounted there in order

to save power. Both light sources are controlled by the main controller via a digital signal. The encoder motors are controlled via the motor driver installed on the PCB

(TB6612FNG). To use the encoder, an interrupt-driven PD regulator must be implemented in software. Our main controller also reads the single binary switch on the handle, which is used to start and stop the robot. In order to



successfully navigate around obstacles and locate the entrance or exit of the



evacuation zone, four VL53L0X time-of-flight laser distance

sensors were installed. The configuration consists of one sensor on each side and two at the front. One sensor is oriented in a straight line, while the other is positioned at a specific angle. They have

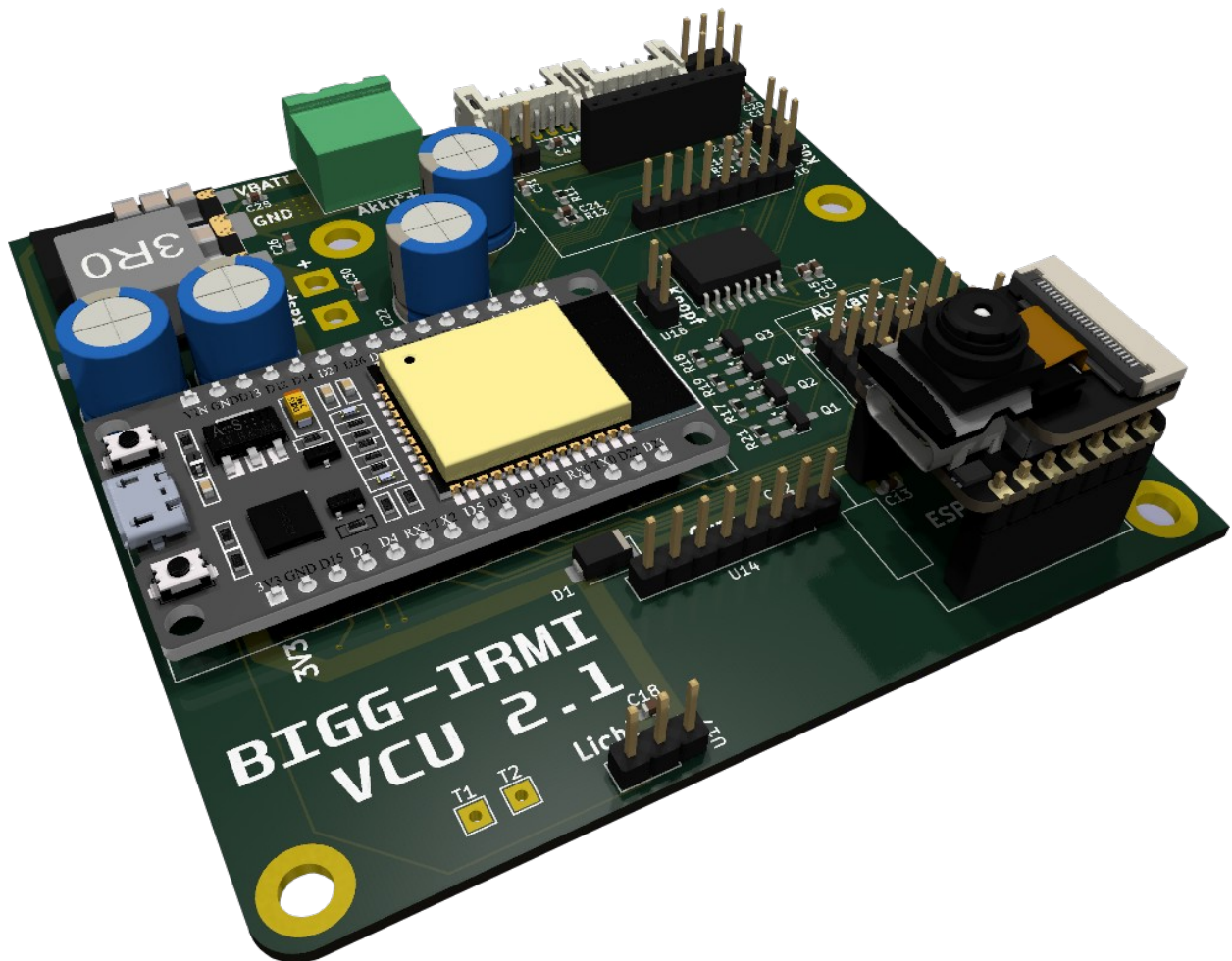
demonstrated consistent reliability in competitive environments and are highly precise. All of the sensors are used to measure precise obstacle dimensions for optimal avoidance and reliable exit detection. We have chosen a 3-cell lithium-ion battery with a total capacity of 2,200 mAh at 12 V to power our robot. This battery can be fully charged in under an hour thanks to its quick-charging capabilities. This is essential as we need to carry out lots of tests in competition environments and the battery only lasts for around 30 minutes.



To prevent any unexpected crashes due to low voltage, we have installed a voltage divider to enable our main controller to read the battery voltage and signal low voltage via the installed lights. Our victim rescue device is equipped with two light sensors. Each sensor features a simple voltage divider and filtering circuitry to reduce noise levels, ensuring a consistent and reliable voltage output for the ADCs in our main controller. Our rescue system also utilises a small MG90 metal geared micro servo to tighten the cord, as well as a large metal geared high torque lifting servo from Makeblock. Extensive testing over many years has



demonstrated the robustness, reliability and power of the servos. Both servos are controlled by the main controller via digital pulse width modulation (PWM). To detect ramps and enable precise turning, we installed an MPU6050 9-axis gyro sensor, which is connected to the main controller via I2C. We also fitted a 12V cooling fan to the camera controller and motor driver, as these components get quite hot during normal operation.



Software

Architecture design

Our software is made up of several elements, each of which runs on a separate microcontroller or single board computer (SBC). Each element contains our custom-written HLDC byte-stuffing UART communication class, enabling it to easily send motor commands, read sensor values, and control servos. All programs are written in C++, except for the SBC code, which is written in Python.

Overview of the elements:

- Line tracking, intersection handling and end detection (ESP32S3)
- Evacuation Zone entry detection and Victim & Evacuation Point location (Raspberry Pi 5)
- Sensor interface, motor control, lighting and “command centre” (ESP32)

Line tracking, intersection handling and end detection

The program operates using the primary front camera (ESP32S3), which contains a bespoke line tracking algorithm that has been developed from ground up. A variety of sophisticated techniques were employed to optimise runtime performance. These optimisations enable us to process approximately 20 FPS (30 ms capture, 29 ms decode, 800 μ s line tracking). For further information, please refer to the Innovative Solutions section.

Evacuation zone entry detection and victim & evacuation point location (AI)

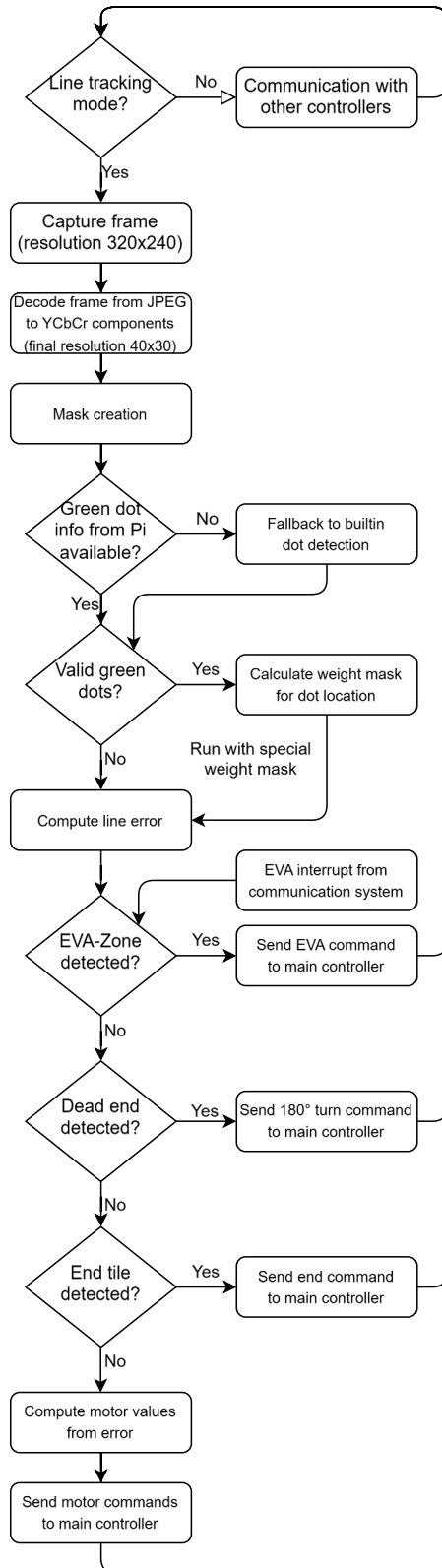
It runs on our sole SBC: the Raspberry Pi 5. It uses the front auxiliary and only rear cameras. For evacuation zone entry detection, it uses a locally running, custom-trained AI on a 128×128 RGB frame at approximately 30 FPS. The system can detect the silver tape at the entrance to the evacuation zone and differentiate between the entrance and exit during an exit search. Victim and evacuation point location also uses a different custom-trained local AI running on a 128×128 RGB frame at approximately 30 FPS. The system can identify live and deceased victims, as well as evacuation points, within the evacuation zone. As both AI outputs are noisy, complex software post-processing is necessary. All AI models are based on YOLOv8 and are trained locally on a powerful GPU using a dataset totalling over 4,000 manually labelled images.

Sensor interface, motor control, lighting and “command centre”

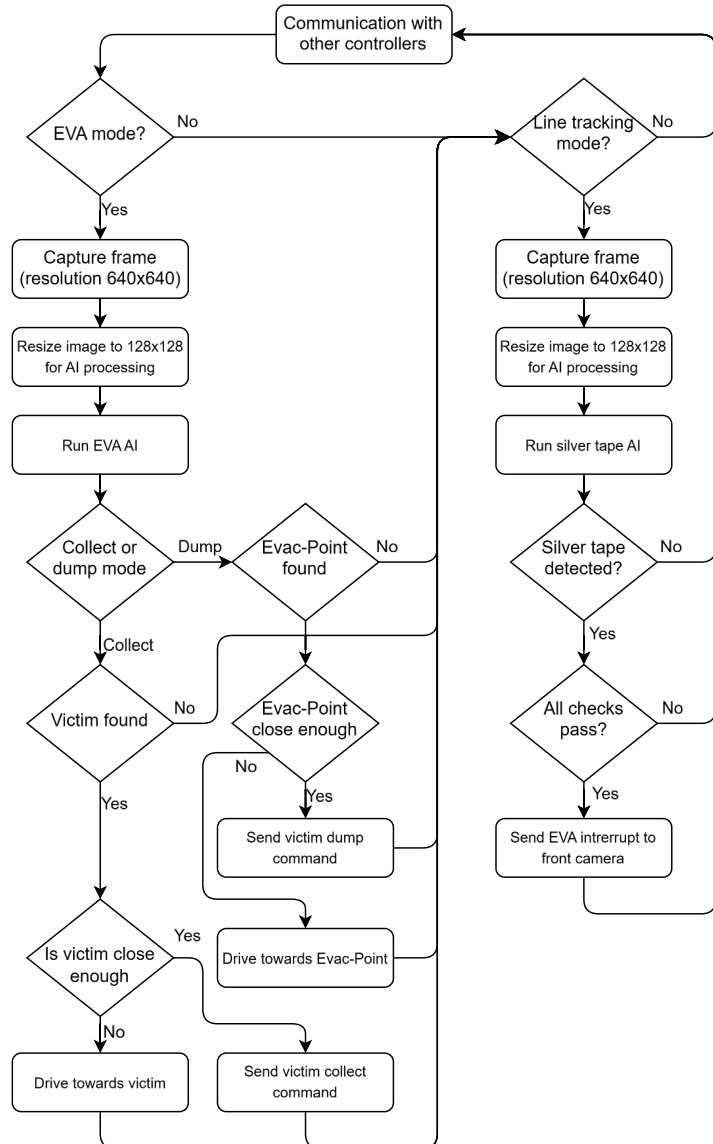
It runs on the ESP32 main controller, which controls the motors, the victim presence sensor, the four VL53L0X distance sensors and the two servos. It also monitors the battery voltage and health of the ESP32S3 and SBC. During operation, it receives commands from the ESP32S3 or SBC and carries out tasks such as turning on the lights, measuring distances and controlling the motors. In line tracking mode, it constantly checks the front distance sensor for obstacles. It also handles start/stop commands given by the button at any time. Additionally, a watchdog is employed to ensure safety and optimal performance during operation.

Flow Charts for each software element

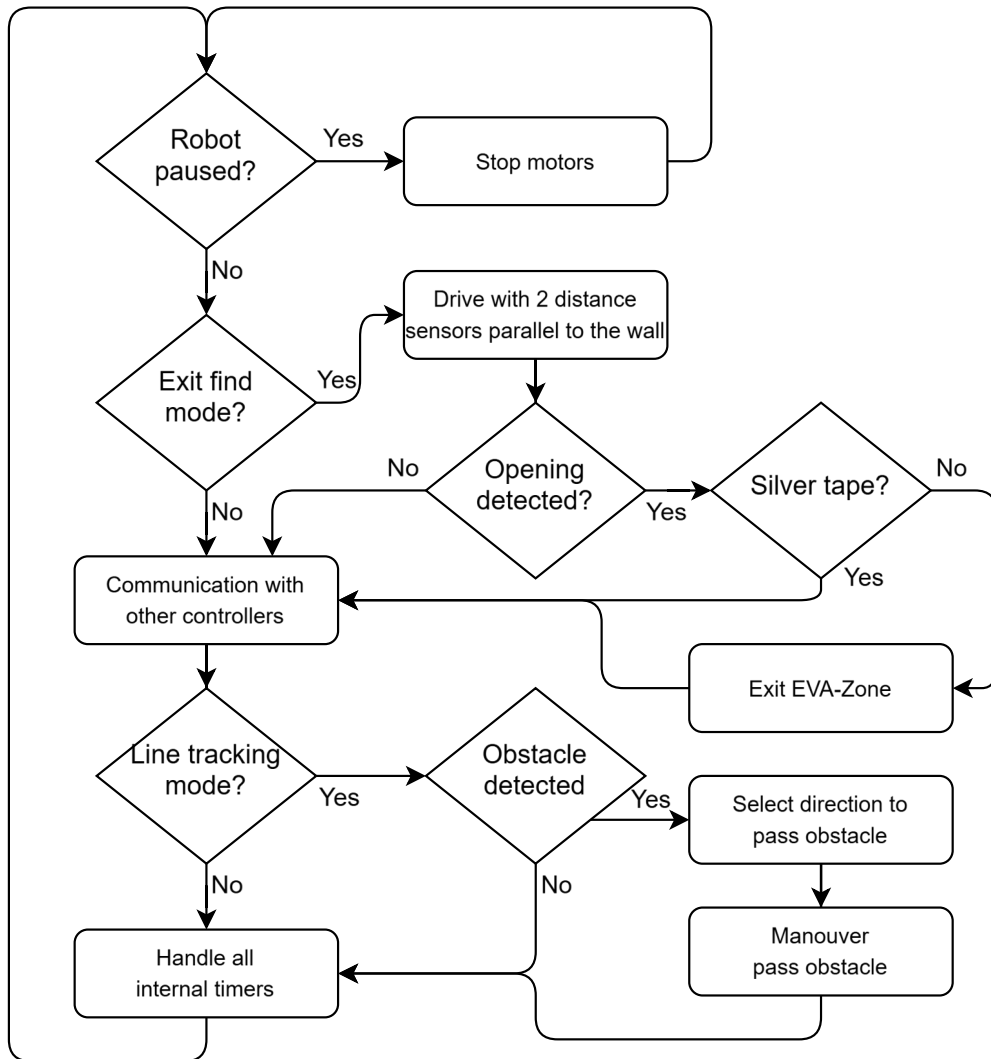
ESP32S3 (line tracking)



Raspberry Pi (AI)



ESP32 (main controller)



Innovative Solutions

Custom line tracking algorithm

Our robot follows the line using a custom algorithm that only works on the currently captured frame. It is evident that a direct comparison between frames is not applicable. The algorithm is first given a 40×30 pixel frame of YCbCr data. The Y component is then used to apply a thresholding algorithm, creating a boolean mask of 40×30 line data. Secondly, it corrects some errors (00100 > 00000 or 11011 > 11111) to provide the main part with clean data. The primary process involves iterating over each pixel to mark the centre of the line in a given row. The coordinates of the centre pixel are used to read it from a weight mask, after which the error value is applied. The weight mask remains constant, with the robot loading different versions of the mask depending on the situation. Lines not pointing in the direction indicated by the green dots will be ignored. In order to ensure the smooth handling of intersections, a calculated factor is applied to the error.



Distance based pixel weight mask (green is 1.0, black is 0.0)

AI-based evacuation zone detection

We have been facing serious challenges in detecting the evacuation zone for some time now. However, we have now implemented an AI-based detection system to address this issue. The AI, which is based on YOLOv8 and is self trained with over 3000 hand labelled images, is capable of processing a 128×128 pixel RGB image at a rate of approximately 30 images per second. This processing speed is more than adequate, even when tracking the line at high speeds, due to the 160° lens of the camera providing a wide field of view. Our AI model has been specifically trained to detect the silver stripe and is now based on two years of data. We can now detect the entrance to the evacuation zone in most cases.

Custom low latency inter-circuit communication protocol

As our robot is split into three separate controllers, with the main controller responsible for controlling all the actuators, we needed a highly reliable, fast and error-free method of communication between the controllers. We then opted for an industry standard: HLDC byte-stuffing. Although there were quite a few libraries available, none of them met our requirements, especially since we needed them to work in both C++ for our ESPs and Python for our Raspberry Pi. Therefore, we decided to develop our own HLDC byte-stuffing library to meet our requirements. It uses various techniques to improve speed and low latency. Now that it is fully developed and set up, we can simply memcpy the data structures from one controller to another in our robot. This solution ensures optimal line tracking responsiveness, which is definitely needed at high speeds.

Quality assurance

To ensure the quality of our software, we have implemented an automated check that verifies communication between controllers and the latencies of every running task, as well as hardware functionality checks. To verify other components of the software, we create test scenarios to evaluate various aspects of the code, e.g. 20 different scenarios for obstacles on ramps. To streamline the debugging process, we have incorporated optional serial printouts. We do not utilise JTAG debugging as this would negatively impact our line tracking performance due to the reduced available processing power. Finally, we perform full test runs to ensure that everything is functioning correctly.

Performance evaluation

In order to evaluate the performance of our robot, we conduct weekly tests to assess new parts and modules. These tests include multiple full runs, as well as extensive testing of newly installed components or newly developed software. In order to simulate a competitive environment, we systematically alter the lighting conditions between tests and introduce increasingly complex tiles until our robot reaches its limit. We then implement improvements and continue to challenge it. In addition, during the course of competitions, we analyse the solutions developed by other teams. A selection of these solutions are then incorporated into our own development process.

Conclusion

After a year of developing this robot, we can say that we have improved in many areas. We reached many of the goals we set ourselves after last year's season. We invested greater effort in the planning and conceptual design of our robot. Our development process was much more elaborate, and we planned, prototyped and tested more. The robot's design is also much more mature and reliable. The robot hardware is more straightforward to work on because each area (e.g. controller, electronics and drivetrain) has its own layer in the robot. The software and electronics have also become much more robust due to the integration of new camera systems and a new controller architecture. We are anticipating this year's Robocup season with interest. It will provide an opportunity to assess the performance of our robot in a competitive environment, as well as facilitating dialogue with other teams and the exchange of ideas regarding their mechanical and software solutions.